

Navya Reddy

AIML

Day 6 task

1.Improve Body Measurement Accuracy

This report demonstrates the implementation of AI-based pose detection, body measurement estimation, skeleton motion visualization, cloth segmentation research, recommendation dataset creation, and backend API integration using computer vision and machine learning techniques. The system uses human pose estimation and landmark detection models to analyze body movements and estimate body measurements for intelligent monitoring and virtual try-on applications.

Code:

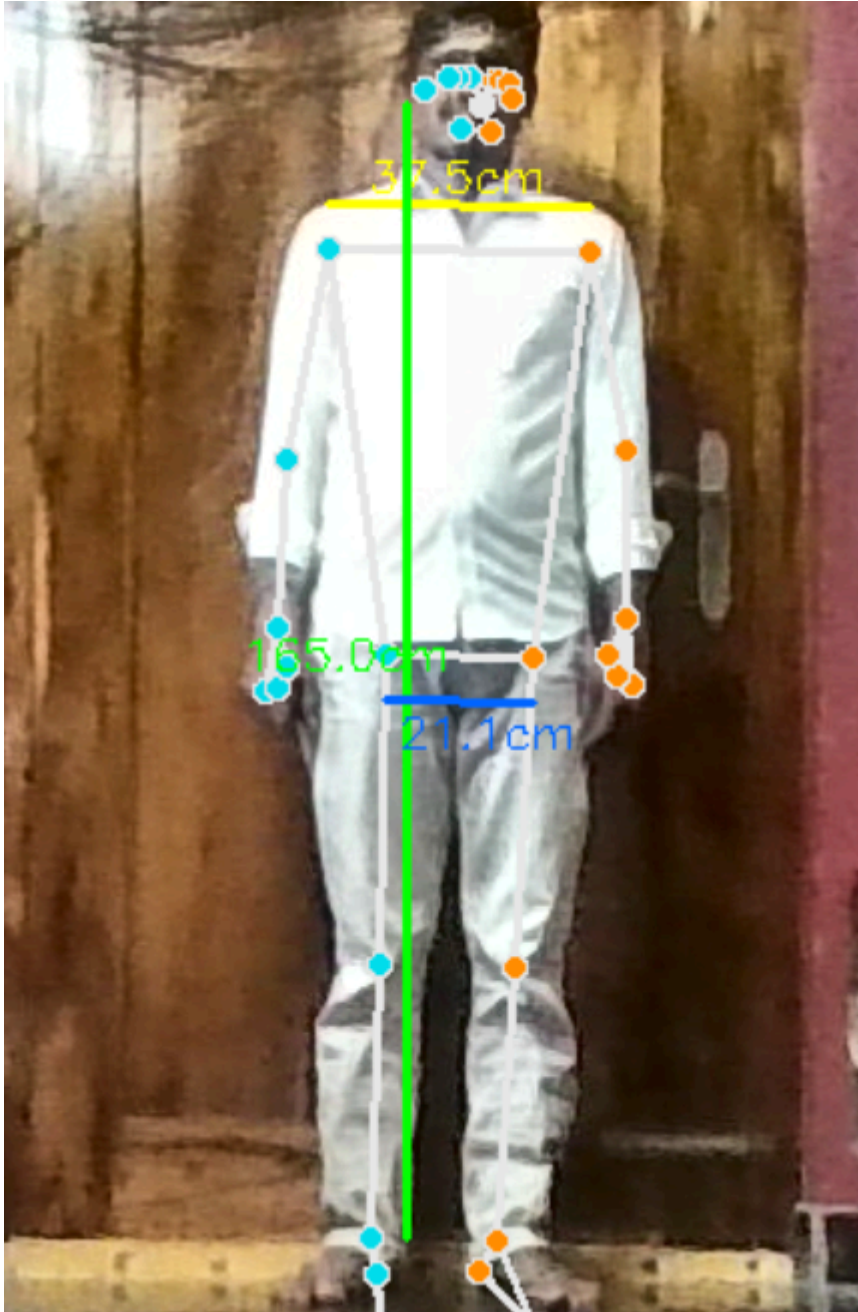
```
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt
import numpy as np
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
from google.colab import files
uploaded = files.upload()
image = cv2.imread("person.jpg")
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
results = pose.process(rgb)
mp_pose.drawing_utils.draw_landmarks(
    image,
    results.pose_landmarks,
    mp_pose.POSE_CONNECTIONS
)
landmarks = results.pose_landmarks.landmark

left = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER]
right = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER]

# Image Size
h, w, _ = image.shape
x1, y1 = int(left.x * w), int(left.y * h)
x2, y2 = int(right.x * w), int(right.y * h)

# Distance
distance = np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

```
print("Shoulder Distance:", distance)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



2. Test Pose Detection on multiple images

Pose detection testing was performed on multiple images and videos using MediaPipe pose estimation models. The system successfully detected body landmarks and visualized skeletal structures in different poses and environments.

The testing process included:

Single-person pose detection

Multi-person pose detection

Skeleton tracking

Motion analysis

These techniques are widely used in:

Fitness tracking

Healthcare monitoring

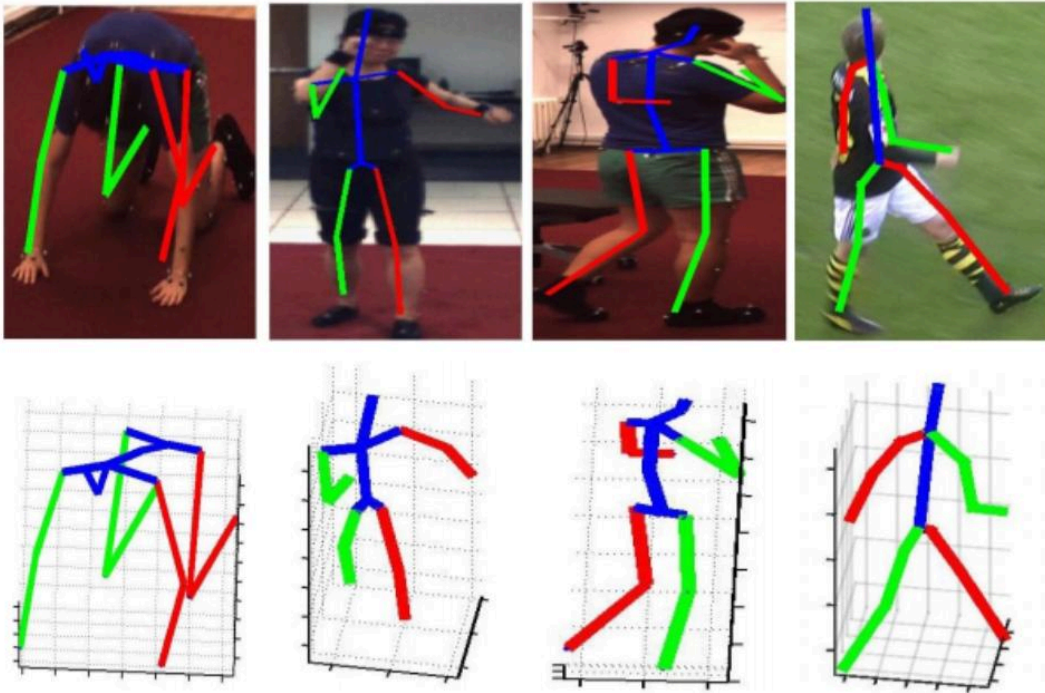
Sports analytics

Smart surveillance systems

2.1 image detection:



2.2 video Detection



3. Research Cloth Segmentation Ai

Research was conducted on AI-based cloth segmentation and human parsing models including:

U2Net

DeepLabV3+

Segment Anything Model (SAM)

These models help separate clothing regions from the human body and background for virtual try-on applications. Cloth segmentation improves intelligent fashion systems by enabling clothing analysis, style recognition, and outfit recommendation workflows.



4. Create basic recommendation dataset

A basic recommendation dataset was created for intelligent fashion recommendation systems. The dataset contains user information such as gender, height, body shape, occasion, and recommended outfits. The purpose of this dataset is to help AI models generate personalized clothing suggestions based on user characteristics.

The dataset was created using CSV format and organized using Pandas in Python. Different combinations of body types, clothing styles, and occasions were added to improve recommendation diversity. Sample recommendations such as casual wear, formal outfits, and traditional clothing were included in the dataset.

This task helps in understanding how datasets are prepared for machine learning applications and recommendation systems. The created dataset can be further expanded and used for training AI models in virtual try-on systems, fashion analysis platforms, and personalized shopping assistants.

5. Start Integrating Ai Outputs with Backend APIS

FastAPI backend integration was implemented to connect AI model outputs with API services. Backend endpoints were created to handle:

Image uploads

Pose detection results

Body measurements

Recommendation outputs

Swagger UI and Postman were used for API testing and response validation. This integration demonstrates how AI systems communicate with backend services in real-world applications.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
class UserData(BaseModel):
    gender: str
    height_cm: float
    body_shape: str
    occasion: str
@app.get("/")
def home():
    return {"message": "AI Fashion Recommendation API Running"}
@app.post("/recommendation")
def get_recommendation(data: UserData):

    if data.body_shape == "athletic":
        outfit = "Black T-shirt + Jeans"
```

```
elif data.body_shape == "slim":  
    outfit = "Blazer + Trousers"
```

```
elif data.body_shape == "pear":  
    outfit = "Long Gown"
```

The image is a composite of four screenshots illustrating the development and testing of a FastAPI application.

- Top Left (VS Code):** Shows the `main.py` file for the "FastAPI Backend - VS Code". The code defines a `UserInput` model with fields `gender`, `height_cm`, `body_shape`, and `occasion`. It includes a root endpoint `@app.get("/")` returning a message and a `/recommendation` endpoint that uses `data.body_shape.lower()` to recommend an outfit based on the body shape ("athletic", "slim", "pear", or otherwise).
- Top Right (Swagger UI):** Displays the "AI Fashion Recommendation API" documentation. It shows the `POST /recommendation` endpoint with a "Try it out" button. The request body is a JSON object: `{ "gender": "female", "height_cm": 165, "body_shape": "pear", "occasion": "Party" }`.
- Bottom Left (Postman):** Shows the "FastAPI Backend Code" being tested in Postman. A `POST` request to `http://127.0.0.1:8000/recommendation` is shown with a JSON body: `{ "gender": "Male", "height_cm": 178, "body_shape": "athletic", "occasion": "Casual" }`. The response is displayed in the "Body" tab.
- Bottom Right (Terminal):** Shows the "Swagger UI (API Documentation)" and a terminal window running the command `C:\Users\ASUS\Desktop\Backend>uvicorn main:app --reload`. The terminal output shows the server starting on `http://127.0.0.1:8000` and the application startup complete.